

IMPLEMENTATION_PLAN_v2

HelixLLM Local-Serving Full-Extension — Implementation Plan v2

Multi-Instance / Multi-Agent Concurrency on RTX 5090 (32 GB, Blackwell sm_120)

Scope: extends 01_local_models_serving.md (2026-07-06) with the operator's 2026-07-08 guidance for heavy multi-instance / multi-agent CLI work (2–3 OpenCode instances × 3–4 sub-agents = 6–12 concurrent sequences), reconciled against the model/serving research already landed AND the already-running production server. This document does **not** repeat content from 01_local_models_serving.md that remains valid — it is cited, not copied. Where the operator's 2026-07-08 guidance is superseded by newer/better-verified information, that is called out explicitly (§11.4.150 deep-research discipline).

Author note (§11.4.6): every claim below is either (a) drawn from 01_local_models_serving.md §8's already-cited sources, (b) drawn from docs/research/07.2026/00_master/RESUME.md's live, operator-testable state, or (c) a NEW citation from this session's research, listed in the Sources footer. Anything not directly evidenced is marked UNCONFIRMED.

Revision note (2026-07-08, post-independent-review, §11.4.134 iterate-to-GO): an independent review (scratchpad/review_serving_plan_v2.md) found 3 Important + 1 Minor finding against the first draft of this document — a factual over-claim about DeepSeek-Coder-V2-Lite's 2026 discourse presence, an under-researched dismissal of Mistral-Nemo-12B, an unreconciled VRAM/KV budget (theoretical ceiling vs. an unverified “observed ~8 GiB free” figure that predated the vision-serving container's teardown), and a reused-not-re-derived MoE/dense KV-ratio imprecision. All four are corrected in place below (§0's reconciliation table, §1(c)'s Lane-B candidate list and Headroom bullet, §1(f) new subsection, and Task 2.1's acceptance criteria), each citing fresh 2026-07-08 sources plus a live read-only nvidia-smi check. See those sections for the corrected content; this note exists so the revision history is visible at the top of the document rather than only in the diff.

0. Reconciliation — what’s already decided vs. what changes

Topic	Already decided (RESUME.md / 01_local_models_serving.md)	Operator’s 2026-07-08 guidance	Reconciliation
Primary coder model	Qwen3-Coder-30B-A3B-Instruct , Q4_K_XL GGUF, LIVE AND PROVEN at http://localhost:18434/v1 (container helixllm-coder), 8 parallel slots, 24k ctx, q8_0 KV, ~19.4 GiB VRAM resident. Measured: ~220 tok/s single-stream, 8 concurrent agents @ 85–96 tok/s each, real coding output validated (docs/qa/phase2_e2e_20260706/).	Qwen2.5-Coder-32B-Instruct Q4_K_M/ Q5_K_M (~20–24 GB) “for coding”	KEEP the running Qwen3-Coder-30B-A3B as primary MoE (3.3B active) vs. Qwen2.5-Coder-32B dense — direct config.json-derived computation this session (Qwen2.5-Coder-32B: 64 layers × 8 KV heads × head_dim 128 → 4096 KiB/token at q8_0; Qwen3-Coder-30B-A3B: 48 layers × 8 KV heads × head_dim 128 → 4096 KiB/token at q8_0) gives a ratio of 48/128 = 0.375 (~3⁄8) , not ~1⁄4 (0.33) as an earlier draft of reconciliation stated — corrected per the independent review (2026-07-08); directionally unchanged (MoE meaningfully smaller — fewer KV heads, fewer layers), weight is still the deciding factor for 12 concurrent sequences. Qwen2.5-Coder-32B remains documented low-parallelism “raw quality” fallback (\$5B dense launch command already exists). No re-download/reinstall needed for the primary model — this is a config-tuning + multi-instance task, not a model-swap task.
Agent/reasoning model	Qwen2.5-32B-Instruct “for agents”	—	Qwen3-32B (already ranked in the existing table, BFCL 75.7%) is the verified, current dense agent/tool-use model on this hardware; Qwen2.5-32B

Topic	Already decided (RESUME.md / 01_local_models_serving.md)	Operator's 2026-07-08 guidance	Reconciliation
			Instruct is its predecessor and is superseded. Recommend Qwen3-32B if a dense low-parallelism agent lane is wanted; keep MoE Qwen3-Coder-30B-A3B as the default agent+coder model since HelixCode's agent loop and coder loop are currently using on one endpoint.
Structured tool-call model	DeepSeek-Coder-V2-Lite (MoE), Mistral-Nemo-12B, Hermes-2-Pro-8B	Second/auxiliary instance for structured output	Corrected 2026-07-08 per independent review — the three are 2024-superseded, not implement” verdict was WRONG for two of the three. Hermes-2-Pro (Mistral-7B-l) genuinely reads as superseded — no 2026 “best local LLM” coverage found for it in this the prior session’s research (Hermes 4 is the actively-promoted successor); this part of the verdict stands. DeepSeek-Coder-V2-Lite and Mistral-Nemo-12B are RETRACTED from the “superseded, do not implement” bucket — both appear in current, dated 2026 discourse and are genuine B candidates on the merits (VRAM footprint + concurrent fit); see §1(c)/§1(d) below for re-evaluation and §2’s reconciliation of the VRAM budget these candidates may fit. Do not implement Hermes-2-Pro as-is; DeepSeek-Coder-V2-Lite and Mistral-Nemo-12B are live Lane-F

Topic	Already decided (RESUME.md / 01_local_models_serving.md)	Operator's 2026-07-08 guidance	Reconciliation
			candidates pending Task benchmarking spike.
VRAM/ concurrency math	§4 of 01_local_models_serving.md — full KV-cache-per-token formula + tables for both dense-32B and Qwen3- Coder-30B-A3B (MoE).	“VRAM/KV- cache budget math for concurrency”	Formula and tables already and are correct and current this plan reuses them verbatim and extends with the current running config's actual numbers (24k ctx, 8 slots, KV, 19.4 GiB resident) as the concrete baseline to scale from
Serving stack	vLLM (source build, sm_120) PRIMARY per the existing doc; llama.cpp FALLBACK, chosen for ease-of-Blackwell-build. HelixLLM's actual production choice: llama.cpp (llama-server via containers submodule, §11.4.76/§11.4.161), consistent with the operator directive here ("Serving stack = llama.cpp in a container via our containers submodule").	llama-server flags	Reconciled: the operational reality is llama.cpp , not vLLM — HelixLLM has never built vLLM. This plan is scoped entirely to llama.cpp/llama- server. vLLM remains a documented but UNADOPTED alternative in 01_local_models_serving.md §3; no work item in this plan touches vLLM.

1. Deep multi-angle web research (2026-07-08) — new findings that extend §01

(a) Best coding+agentic GGUF models for 6–12 concurrent sequences on 32 GB — confirmed + one addition

No change to the top-2 ranking (Qwen3-Coder-30B-A3B primary, Qwen2.5-Coder-32B quality-fallback, Devstral-Small-2507 agent-specialist) — all three remain current per 01_local_models_serving.md §2/§8, re-confirmed this session via Unsloth's docs and the QwenLM/Qwen3-Coder GitHub repo (both accessed 2026-07-08).

Tool-calling reliability update (NEW, this session): an OpenCode GitHub issue (#1809, anomalyco/opencode) reports Qwen3-Coder-30B-A3B tool-call argument type-validation failures (arrays passed as strings) on OpenCode 0.4.12. Unsloth's

documentation states this class of tool-calling bug was **fixed** in their GGUF re-uploads (“tool-calling for Qwen3-Coder was fixed, allowing seamless tool-calling in llama.cpp, Ollama, LMStudio, Open WebUI, and Jan” — Unsloth docs, accessed 2026-07-08). HelixLLM’s running container already sources the Unsloth GGUF (RESUME.md / prior 01_local_models_serving.md §5B: -hf unsloth/Qwen3-Coder-30B-A3B-Instruct-GGUF). **Action item (Phase 1, task 1.1):** verify the exact GGUF file/revision in the running container matches or postdates Unsloth’s fix commit; if not, re-pull. This is a genuine new risk surface for the 6–12-concurrent-agent goal: a tool-call type-validation failure under load is exactly the failure mode that would make “12 concurrent OpenCode subagents” flaky.

New candidate model — GLM-4.7-Flash (30B-A3B MoE) (NEW, this session, NOT in the prior doc’s confirmed table — the prior doc only saw “GLM-4.7” as an UNCONFIRMED aggregator claim). Primary sources this session (Unsloth’s dedicated docs/models/glm-4.7-flash page + the unsloth/GLM-4.7-Flash-GGUF Hugging Face repo, both accessed 2026-07-08) confirm this is a **real, documented** model: 30B total / 3B active MoE (same class as Qwen3-Coder-30B-A3B), 128K native context (200K claimed in some docs — UNCONFIRMED exact number, treat 128K as the citable floor), 4-bit quant $\approx$ 17–18 GB weights. This is architecturally the closest available alternative to the running primary and is worth a **benchmarking spike** (Phase 2) as either (i) a coder-lane alternative if Qwen3-Coder tool-calling issues recur under the 6–12-agent load, or (ii) the second/auxiliary instance model (its MoE profile gives it a small KV footprint, good for a lightly-loaded second lane). Distinguish explicitly from “GLM-5” and “GLM-4.7” (non-Flash, full-size) which remain UNCONFIRMED / do-not-fit-32GB per the prior doc.

(b) llama-server multi-sequence config — exact flags + KV budget math per concurrency level

Confirmed from 01_local_models_serving.md §4/§5B: --parallel, --cont-batching, --cache-type-k/v q8_0, -fa on, --ctx-size = **total** KV budget shared across --parallel slots. New details this session (llama.cpp server README + GitHub Discussion #22658 + #18308 + #4130, all accessed 2026-07-08):

- **--kv-unified / -kvu vs --no-kv-unified / -no-kvu** (this flag is either new or was not previously documented in the existing research; UNCONFIRMED whether it existed at the time of the 2026-07-06 doc). Default appears to be **enabled** when slot count is auto.
 - **--kv-unified** (default): a single shared KV buffer covers the full --ctx-size; slots borrow from the pool dynamically. Better for **heterogeneous request lengths** — e.g. one subagent doing a short tool-call round-trip next to another doing a long context-stuffed review — because a short request doesn’t waste a full static slice.

- `--no-kv-unified`: `--ctx-size` is **statically split** across `--parallel` slots (`--ctx-size 100000 --parallel 4` → each slot gets a fixed 25k). Predictable per-slot ceiling, but a burst of short requests can't borrow a starved slot's spare capacity.
- **Recommendation for the 6–12-agent goal**: use `--kv-unified` (the default) rather than the currently-configured implicit static-split behavior, because CLI-agent workloads are inherently heterogeneous (short tool round-trips interleaved with long context turns). **Action item (Phase 1, task 1.2)**: confirm which mode the running `helixllm-coder` container is actually in (explicit flag not currently visible in `RESUME.md`'s recorded launch command — the original launch predates this flag's discovery) and switch to `--kv-unified` explicitly if not already default, with a measured before/after comparison.
- **Prefill vs. decode slot starvation** (llama.cpp Discussion #4130, markaicode.com architecture writeups, accessed 2026-07-08): the 3.8× continuous-batching throughput gain over sequential decode holds **only when prompts are short** (<256 tok cited for one benchmark); long prompts monopolize the prefill phase and can starve decode slots serving other agents. This is a concrete, previously-undocumented **danger zone** for a system-prompt-heavy multi-agent CLI workload (agent framework system prompts + tool schemas commonly run several hundred to several thousand tokens). Mitigated by prefix caching (already noted in the existing doc for vLLM; llama.cpp's analogous mechanism is **prompt-slot-reuse / KV-cache-reuse**, GitHub Discussion #13606, accessed 2026-07-08) — a request whose prefix matches a cached slot's prior prompt skips re-prefilling that prefix.
- **--batch-size / --ubatch-size**: increasing `--parallel` without increasing prefill-tokens-per-pass via `--batch-size/--ubatch-size` “spreads the same throughput across more queues — latency goes up, aggregate tokens/sec does not” (markaicode.com, accessed 2026-07-08). **Action item (Phase 1, task 1.3)**: the currently-running 8-slot config's `--batch-size/--ubatch-size` values are not recorded in `RESUME.md`; capture them and tune upward if raising `--parallel` toward 12–16 slots (Phase 2) without a matching batch-size increase would be a false optimization.
- **/metrics endpoint** (llama.cpp server README, re-confirmed): exposes tokens/sec, KV cache usage, and queue depth — this is the concrete mechanism for Phase 3's runtime-signature dashboards (§11.4.108).

(c) Single- vs. multi-instance VRAM partitioning

Confirmed general practice (aimadetools.com, sitepoint.com “Multiple Local Models” — accessed 2026-07-08): run **independent llama-server processes**, each bound to a distinct port, with per-model `-ngl/--ctx-size` chosen to partition VRAM deliberately; monitor with `nvidia-smi/nvtop` to catch silent CPU fallback

(a model whose weights partially spill to system RAM because VRAM ran out — this degrades silently rather than erroring, a genuine danger zone, see §3 below). No native “shared-process, two-model” mode exists in llama.cpp comparable to vLLM’s multi-model serving — **each model is its own container + process**, consistent with HelixLLM’s existing per-class container pattern (helixllm-coder, imagegen-boot, visiongen-boot, videogen-boot in cmd/).

Recommended partition for the 32 GB card (building on the reconciled model choices in §0 and the measured 19.4 GiB resident baseline): - **Lane A (resident, always-on)**: Qwen3-Coder-30B-A3B, ~19.4 GiB — unchanged, ClassCoder (broker.go: IsResident() == true, never evicted, never counted against admission). - **Lane B (warm, admission-gated)**: a second small instance for structured/tool-heavy or overflow agentic work, ClassVLM or a new warm class (see §4 broker integration below). **Candidate list corrected 2026-07-08 per independent review** — two candidates dismissed in an earlier draft as “2024-superseded” are RETRACTED from that dismissal and reinstated below with real VRAM footprints, cross-checked against the corrected budget in the Headroom bullet immediately following this list: - **(1) Mistral-Nemo-12B** at Q4 (Q4_K_M ≈ 7.0 GB decimal ≈ 6.52 GiB weights — willitrunai.com, mljourney.com, localaimaster.com, all accessed 2026-07-08) — genuinely current 2026 discourse, not 2024-superseded as an earlier draft claimed: ranked #4/15 on pocketllm.app’s 2026 “best local LLM models” listicle (88/100, “the best ‘if you have a gaming laptop’ model”, 128K context, “quantization-aware design that actually works at Q4” — accessed 2026-07-08), and explicitly still recommended in 2026 sources for VRAM-constrained deployment where Mistral Small 3.1 cannot fit. Against this plan’s corrected ~10.39 GiB Lane-B budget (see Headroom bullet below), 6.52 GiB weights leaves ~**3.87 GiB for KV cache + activation buffers** — a genuinely healthy margin for a lightly-loaded second lane. **Promoted to top priority** on VRAM-fit merits (largest KV headroom of any candidate here), pending Task 2.1’s tool-calling-correctness benchmark (function-calling support must still be verified against this plan’s actual tool-schema, not merely cited from third-party listicles). - **(2) GLM-4.7-Flash** at its smallest available quant (UD-IQ2_XXS, ~10.5 GB decimal ≈ 9.78 GiB weights — confirmed against the Unsloth quant table) IF the coder lane’s tool-calling under load proves unreliable and GLM-4.7-Flash benchmarks better on that axis specifically. **Explicit go/no-go, corrected 2026-07-08 (see Headroom bullet)**: at this quant, 9.78 GiB weights against the ~10.39 GiB budget leaves only ~**0.61 GiB for KV cache + activation buffers** — this does NOT support the “lightly-loaded second lane serving multiple concurrent sequences” rationale as originally stated; realistically it supports only a single low-context slot (--parallel 1, small --ctx-size), not concurrent serving. Do not select this candidate for the “light concurrent second lane” role at this quant without re-running the go/no-go math against whatever quant is actually benchmarked. - **(3) DeepSeek-Coder-V2-Lite** (MoE, 16B total / 2.4B active) at Q4_K_M (10.36 GB decimal ≈ 9.65 GiB weights, bartowski/DeepSeek-Coder-V2-Lite-Instruct-GGUF HF

page, accessed 2026-07-08) — **RETRACTED from “does not appear in current 2026 discourse”** (an earlier draft’s claim was checked this session and found false): ranked #7/15 on pocketllm.app’s “best local LLM models 2026” listicle (85/100) and #2/8 on the same site’s dedicated “best local LLM for coding 2026” listicle (91/100, “the best local coding model that fits on a consumer laptop”, “mid-80s HumanEval”, both accessed 2026-07-08), consistently described as “a mixture-of-experts model where only 2.4B parameters are active per token, so it runs at roughly 3B speeds while holding 16B quality.” Against the ~10.39 GiB Lane-B budget, 9.65 GiB weights leaves **~0.74 GiB for KV cache** — the same squeeze class as GLM-4.7-Flash’s smallest quant (2). A genuine Lane-B candidate on model-quality/architecture merits, but under the current VRAM budget it likewise supports only a single-slot, small-context deployment, not a multi-sequence “light concurrent lane,” unless a smaller quant (Q3/Q2) is benchmarked — sizes for those quants were not located this session and are **UNCONFIRMED**, flagged as a Task-2.1 follow-up rather than guessed. - **(4) a second Qwen3-Coder-30B-A3B replica** at a lower quant/ctx (MoE weight-sharing across the two processes is NOT possible in llama.cpp per the vLLM-comparison research above — each process loads its own copy — so a second full replica costs roughly another ~18 GB, which does **not** fit the corrected ~10.39 GiB budget below without a much more aggressive quant/ctx reduction than previously stated; demoted from its earlier “tight but survivable” framing now that the real free-VRAM figure is in hand — see Headroom bullet). - **(5) Hermes-2-Pro-8B’s structured-output role remains unfilled** — this session’s research did NOT find a citable, current, small (≤ 8 B) model that clearly supersedes Hermes-2-Pro for pure JSON/structured-output work specifically (as opposed to general coding/agent use, which candidates (1)–(3) above now cover). **Reconciliation with candidates (1)/(3):** Mistral-Nemo-12B and DeepSeek-Coder-V2-Lite are NOT being proposed as Hermes-2-Pro replacements for pure structured-output — they are proposed as general Lane-B coding/agent overflow candidates, a different role. The ≤ 8 B-structured-output gap remains genuinely **UNCONFIRMED / requires a dedicated follow-up research pass** (do not guess per §11.4.6) — this is a narrower, still-open gap than an earlier draft implied by dismissing all three original candidates together. - **Headroom — corrected 2026-07-08 against a live nvidia-smi read (read-only, this session, no coder disturbance):** `nvidia-smi --query-gpu=memory.total,memory.used,memory.free` reports **32607 MiB total (~31.84 GiB), 19436 MiB used (~18.98 GiB — matches Lane A’s measured ~19.4 GiB resident alone, confirming the separately-designed vision-serving container has since been torn down), 12685 MiB free (~12.39 GiB)**. `broker.go’s HeadroomBytes = 2 GiB` is a hard floor already coded and tested (`admit()` fails closed). Lane B’s real, current budget is therefore **12.39 GiB – 2 GiB headroom ≈ 10.39 GiB** — this is now the authoritative figure for this plan (superseding both the ~8 GiB figure recorded earlier while the vision container was still resident, which is stale and no longer representative, AND the ~10.6 GiB purely-theoretical static-arithmetic ceiling, which this live reading now confirms independently to

within ~0.2 GiB). This ~10.39 GiB figure rules out any dense ≥ 13 B model at 4-bit and (4)'s second-replica option above, and is the exact number every candidate's go/no-go check in (1)–(3) is computed against. **Re-check before Task 2.1 executes** — this is a point-in-time reading (2026-07-08) and VRAM state can change if any other lane boots between now and the benchmarking spike.

(d) Latest 2026 models the operator didn't mention

- **GLM-4.7-Flash** — see §1(a)/(c) above; the one credible NEW addition this session.
- **DeepSeek-V4** (mentioned by remoteopenclaw.com's "Hermes Agent" blog posts, March 2026, 81% SWE-bench Verified, 1M ctx) — **does not fit 32 GB** at any usable quant (it is a frontier-scale model in the same tier as DeepSeek-V3, which the existing doc already excludes for VRAM reasons). Not actionable for this hardware; noted for completeness only.
- **Qwen3.5 / Qwen3.6 / GLM-4.7 (non-Flash) / GLM-5** — remain **UNCONFIRMED** this session, same as the prior doc's finding; no primary model card located. Do not plan around these.
- Aider-leaderboard-class verification of Qwen3-Coder-30B-A3B specifically (vs. its 480B sibling) remains **UNCONFIRMED exact number** — unchanged from the prior doc's honesty note.

(e) VRAM broker gating for multi-instance admission

Read submodules/helix_llm/internal/vrambroker/broker.go + budget.go this session (not new web research — direct source read, §11.4.78 CodeGraph-equivalent). Findings directly relevant to this plan:

- The broker already models exactly the Lane A/Lane B split needed: `ClassCoder` is `IsResident()` (always granted, pinned, uncounted) — this **already matches** the running coder container's actual behavior (it's never admission-gated, it just runs). `ClassVLM` and `ClassTranslate` are "warm tier" — admission-gated on **measured** free VRAM via `nvidia-smi` (`budget.go: readNvidiaSMI`), fail-closed (`admit(): free >= needBytes + headroom`). `ClassImage/ClassVideo` are burst, single-owner (§11.4.119), mutually exclusive.
- **Gap for this plan's Lane B:** there is currently no broker Class for "second coder/agent instance." The broker's Class enum (`ClassCoder|ClassVLM|ClassImage|ClassVideo|ClassTranslate|ClassEmbed`) has no slot for a second LLM-serving lane distinct from the vision-language-model (`ClassVLM`) use. **Design decision needed (Phase 1, task 1.4):** either (i) reuse `ClassVLM` semantically for "any warm-tier auxiliary LLM instance" (cheapest, no broker code change, but semantically confusing — `ClassVLM` literally means vision-language-model per the existing docs/research/07.2026/01_local_models_serving/ vision-serving design), or (ii) add a

new ClassAgent (warm tier, admission-gated like ClassVLM/ClassTranslate) to broker.go's enum + IsBurst()/IsResident() methods. This plan recommends **(ii)** — see Phase 1 task 1.4 for the concrete diff scope.

- The broker's admission math is VRAM-only; it does not currently model **KV-cache growth under concurrent load** (a lane that was admitted at boot with N GB of static weights can still be starved of KV cache at runtime if --ctx-size/--parallel are set too aggressively for the admitted budget). This is captured as Danger Zone D3 below — the broker's job is admission (does the lane fit at all), not runtime KV headroom (does the configured concurrency level fit within what was admitted). These are two different correctness properties and must not be conflated.

(f) Mapping “2–3 OpenCode instances × 3–4 sub-agents” onto serving endpoints/lanes

(NEW subsection, added 2026-07-08 — completeness gap identified by independent review, §5 of the review: the prior draft never stated explicitly whether multiple OpenCode instances share one endpoint or need distinct ones.)

Explicit assumption (stated here for the first time in this plan): ALL 2–3 OpenCode instances point at the **same Lane A endpoint** (<http://localhost:18434/v1>, the resident helixllm-coder container) and **share its single --parallel slot pool** — this plan does NOT propose giving each OpenCode instance its own dedicated llama-server process/port. This follows directly from Lane A being llama.cpp's only per-model process (§1(c): “no native shared-process, two-model mode... each model is its own container + process”) — since all instances want the *same* coding model, they are, by construction, clients of the *same* process, not separate lanes. Lane B (§1(c)/§4) is scoped as a small structured/overflow auxiliary lane for the whole system, NOT a second per-instance dedicated coding endpoint — it does not change this picture.

Concurrency math under this assumption: the operator's stated target is 2–3 instances × 3–4 sub-agents = **6–12 concurrent sequences**, all as concurrent HTTP requests against Lane A's one --parallel slot pool. This is exactly what Task 1.3's --batch-size/--ubatch-size tuning memo and the Phase 4 --kv-unified + batch-size + --parallel (8 → 12–16) changes are sized for — Task 1.3's “12-agent target” sizing memo already targets the upper end of this exact range. If the live --parallel value is below the actual concurrent-sequence count at any moment (e.g. still at 8 slots pre-Phase-4 while 12 sequences are active), llama.cpp's continuous batching **queues** the excess requests rather than erroring (confirmed general behavior per §1(b)'s cited llama.cpp architecture sources) — this is graceful degradation (added latency), not a hard failure, but it does mean the “6–

12 concurrent” goal is not fully met until Phase 4’s --parallel increase actually lands (which is operator-gated, D8).

Action item (folds into existing Task 1.3, no new task needed): Task 1.3’s sizing memo MUST explicitly confirm/deny this shared-single-endpoint assumption is what the operator intends before the Phase-4 batched coder-pause window is requested — if instead the intent is for one OpenCode instance to be routed to Lane A and others to a distinct lane (which would require a second full coding-capable instance, i.e. candidate (4) in §1(c), demoted for VRAM-fit reasons under the corrected budget in §1(c)’s Headroom bullet), that is a materially different, more VRAM-constrained topology and must be confirmed explicitly rather than assumed.

2. Multi-pass danger-zone analysis (§11.4.92 Pass 2/3, §11.4.85 stress/chaos framing)

Eight danger zones enumerated, each with a concrete, testable mitigation. (Count: 8.)

D1 — VRAM OOM under concurrency burst (Lane A + Lane B simultaneous peak)

Mechanism: Lane A (coder, resident) and Lane B (auxiliary, warm) are independently admitted, but `nvidia-smi’s memory.free` read in `budget.go` is a **snapshot at admission time**; a burst where both lanes simultaneously ramp KV-cache usage to their configured ceilings (Lane A’s 8–16 slots all filling their --ctx-size share, Lane B doing the same) can exceed what was true at admission if either lane’s static KV-cache ceiling was configured larger than what was actually free when it was measured. **Mitigation:** (i) Lane A and Lane B’s --ctx-size values MUST be sized so their **worst-case static KV ceiling** (not just weights) sums to $\leq$ card total – headroom, computed via the §4 KV-cache-per-token formula — this is a **config-time**, not admission-time, guarantee and must be checked mechanically (Phase 1 task 1.5: a pre-boot config validator). (ii) Runtime guard: a lightweight sidecar polls `/metrics` on both lanes + `nvidia-smi` every N seconds and alerts/throttles-new-connections if combined VRAM crosses a soft ceiling below hard OOM (Phase 3). (iii) Chaos test: intentionally launch Lane B while Lane A is saturated at 8/8 slots and assert the broker refuses admission (`ErrBudgetExceeded`) rather than the OS OOM- killing a container.

D2 — KV-cache exhaustion under long-context agent turns

Mechanism: per §1(b), a --no-kv-unified static split starves a slot handling an unusually long turn (e.g. a subagent doing a large diff review) once that slot’s fixed

share is consumed — subsequent requests to that slot either truncate context or error, independent of whether other slots are idle. **Mitigation:** switch to `--kv-unified` (§1(b) recommendation) so idle slots' capacity is available to a long-running one; add a `/metrics`-derived per-lane “KV utilization %” runtime signature (§11.4.108) with an alert threshold (e.g. >90% sustained for >30s); stress test (§11.4.85): 12 concurrent synthetic agents, one deliberately sending a near-ctx-limit prompt, assert no other agent's request is refused/degraded.

D3 — Slot starvation from prefill-heavy system prompts (§1(b))

Mechanism: CLI-agent frameworks (OpenCode et al.) resend a large system prompt + tool schema on every turn; if prefix caching / KV-cache-reuse is not actually active (misconfigured or the framework varies the prompt slightly turn-to-turn, invalidating the cache), every turn re-prefills the full system prompt, monopolizing prefill and starving decode for other agents' slots — measured effect per §1(b) is up to $3.8\times \rightarrow 1\times$ throughput collapse if prefill dominates.

Mitigation: (i) verify via `/metrics` (or `llama.cpp`'s cache-reuse debug logging) that the system-prompt prefix is actually being reused turn-to-turn for a real OpenCode session — this is a **runtime-signature** check (§11.4.108), not assumable from config alone. (ii) If the calling framework varies the prompt (timestamps, session IDs injected into the system prompt), this is a **client-side** fix (stabilize the prefix, move variable content to the end of the prompt) — flag as a HelixAgent/OpenCode-integration task, not a llama-server task. (iii) Stress test: replay a captured real multi-turn OpenCode session at 8–12x concurrency and measure prefill-vs-decode time split from `/metrics`.

D4 — Model-swap thrash if Lane B is not truly independent from Lane A

Mechanism: if a future change accidentally makes Lane B share the same llama-server process/port as Lane A (e.g. via a naive “just point OpenCode's second config at coder too” shortcut), the two workloads contend for the SAME slot pool rather than being independently admission-gated — this silently reduces effective coder concurrency without any error. **Mitigation:** Lane B MUST be its own container + port (per §1(c) — this is the only supported llama.cpp topology); a pre-boot config validator (shared with D1's task 1.5) asserts distinct ports/container names for every admitted lane; a Challenge test boots both lanes and asserts two distinct podman ps entries with two distinct listening ports.

D5 — Coder-vs-new-instance VRAM contention at broker level (the ClassVLM reuse gap, §1e)

Mechanism: if Lane B is implemented by reusing ClassVLM (§1(e) option (i)) rather than a new ClassAgent, a future genuine vision-serving workload (the

separately-designed `scratchpad/design_vision_local_serving.md` Qwen2.5-VL-7B work already in `RESUME.md`'s next-actions) would collide with Lane B for the same broker class, and the single-owner-style semantics the broker doesn't currently enforce for `ClassVLM` (it's warm-tier, not burst/ single-owner) would let both be admitted simultaneously even if the card can't actually hold both plus the resident coder. **Mitigation:** implement the new `ClassAgent` (§1(e) option (ii)) rather than reusing `ClassVLM` — this is the concrete reason Phase 1 task 1.4 recommends option (ii). Regression test: admit `ClassAgent` + `ClassVLM` simultaneously at a combined size that exceeds free VRAM and assert the broker's `fail-closed admit()` refuses the second one (this already works generically in `broker.go`'s warm-tier path — the test only needs the new class wired through).

D6 — Tool-calling type-validation failures under load (§1(a), the OpenCode #1809 issue)

Mechanism: a tool-call argument type mismatch (array-as-string) that manifests intermittently is exactly the kind of defect that “looks fine” in a single manual test but recurs under the operator's target load (6–12 concurrent agentic sequences, each issuing many tool calls) — this is precisely the class of defect §11.4.107/ §11.4.146 exist to catch. **Mitigation:** (i) confirm the running GGUF is Unsloth's fixed revision (§1(a) action item); (ii) author a reproduce-first test (§11.4.115/ §11.4.146) that drives N concurrent tool-calling turns against the live `helixllm-coder` endpoint with array-typed tool arguments and asserts zero type-validation failures across all N; (iii) register this as a permanent regression guard (§11.4.135) in `HelixLLM`'s test suite, not a one-off manual check.

D7 — Silent VRAM-to-RAM spillover (CPU fallback) under multi-instance misconfiguration

Mechanism: per §1(c), if Lane B's `-ngl` (GPU layer count) or `ctx-size` is set larger than what actually fits after Lane A is resident, `llama.cpp` can silently offload some layers to CPU RAM rather than failing outright — this produces a working-but-catastrophically-slow Lane B (not a crash, not an error — just an unexplained throughput cliff that would be very confusing to debug without the right signal). **Mitigation:** the pre-boot config validator (D1/D4's task 1.5) MUST also assert `-ngl 99` (all layers on GPU) is achievable given the broker's granted lease size for that lane — i.e. reject a Lane B config whose declared VRAM need doesn't actually cover full-GPU-residency at the configured `ctx/quant`, rather than letting `llama.cpp` silently degrade. Runtime signature: compare `nvidia-smi` VRAM delta after Lane B boot against the model's expected weights+KV size — a mismatch (VRAM grew less than expected) is the CPU-fallback tell.

D8 — Coder container never-restart constraint vs. Lane B iteration

Mechanism: per RESUME.md, the running helixllm-coder container is explicitly “never restart” (operator-authorized coder-pause required, §11.4.122) — this is a hard constraint on THIS plan’s implementation work: any change to Lane A’s launch flags (e.g. the --kv-unified switch recommended in §1(b), task 1.2) requires a container restart, which is an operator-gated action, NOT autonomously takeable. Lane B, being a NEW container, has no such constraint and can be iterated on freely. **Mitigation:** sequence the plan (see Phase ordering below) so that ALL Lane-A-affecting changes (kv-unified flag, batch-size tuning, GGUF-revision re-pull) are batched into a SINGLE operator-authorized coder-pause window rather than several, and everything else (Lane B build-out, broker ClassAgent addition, config validator, regression tests, dashboards) is done first and independently, since none of it requires touching the live coder container.

3. Phased implementation plan

Every phase’s acceptance evidence follows §11.4.108 (one machine-checkable runtime signature per task, verified on a clean/current target) + §11.4.5 (captured proof, not metadata-only). Every task is independently reviewable (§11.4.125/ §11.4.142) and subagent-dispatchable (§11.4.70) — phases are NOT required to run strictly serially; Phase 1 tasks 1.1/1.3/1.4/1.5 and Phase 2’s benchmarking spike are independently startable in parallel per §11.4.103.

Phase 0 — Baseline capture (prerequisite for everything else; no coder-container touch)

Task 0.1 — Capture the exact current helixllm-coder launch config. - Action: `podman inspect helixllm-coder` (or equivalent container-introspection via the containers submodule’s `pkg/health/pkg/compose` primitives, §11.4.76/ §11.4.161 — never a raw `ad-hoc docker inspect`) to extract the exact llama-server argv currently running: confirm/deny presence of `--kv-unified/--no-kv-unified`, `--batch-size`, `--ubatch-size`, the exact GGUF filename+quant, and the exact `--ctx-size/--parallel` pair. - Acceptance (§11.4.108 runtime signature): a captured text file `docs/qa/helixllm_phase0_baseline_<ts>/container_inspect.txt` containing the full argv, committed under `docs/qa/` per §11.4.83. - Why first: every subsequent task (1.1, 1.2, 1.3, D1–D8 mitigations) references “the running config” — this task turns that into a cited fact instead of an assumption (§11.4.6).

Task 0.2 — Capture current per-agent-count throughput baseline (read-only, no config change). - Action: replay (or re-run, if the original harness/log is

available per `RESUME.md`'s `coder_live_e2e_log`) the 1-agent and 8-agent throughput measurements against the live endpoint; if the harness exists at `docs/qa/phase2_e2e_20260706/`, re-run it rather than writing a new one (§11.4.74 reuse-before-reimplement). - Acceptance: `/metrics` snapshot + per-agent tok/s log captured to `docs/qa/helixllm_phase0_baseline_<ts>/throughput_baseline.log`. This is the “before” half of every subsequent before/after comparison in Phase 1.

Phase 1 — Non-coder-container-touching work (parallel-safe, no operator gate)

Task 1.1 — GGUF-revision verification (§1a, D6). - Action: compare the GGUF file's SHA/commit metadata in the running container (from Task 0.1) against Unsloth's `unsloth/Qwen3-Coder-30B-A3B-Instruct-GGUF` HF repo's fix-commit date (Unsloth docs, accessed 2026-07-08 — cite exact commit once identified, currently UNCONFIRMED exact SHA in this session's research, needs a direct HF API/WebFetch check as a sub-task). - Acceptance: a verification report stating either “current revision postdates the fix” (cite both dates) or “current revision predates the fix — re-pull required” (tracked as a Phase-1.5 action gated behind the coder-pause window, D8).

Task 1.2 — `--kv-unified` config validator + recommendation memo (no container change yet). - Action: since Lane A cannot be touched without a coder-pause (D8), this task ONLY produces the config diff + rationale memo + a Lane-B-testable proof-of-concept (boot a throwaway small model, e.g. an already-available small GGUF, with `--kv-unified` vs `--no-kv-unified` and measure heterogeneous-load behavior difference) — the actual Lane A flag flip is deferred to the batched coder-pause window (D8 mitigation). - Acceptance: `docs/qa/helixllm_phase1_kvunified_<ts>/` with the PoC's before/after `/metrics` comparison on the throwaway model + the exact diff to apply to Lane A's launch command.

Task 1.3 — `--batch-size/--ubatch-size` tuning memo. - Action: using Task 0.1's captured current values, compute (via the §4 KV formula + the `markaicode.com` prefill-throughput guidance, §1b) whether raising `--parallel` from 8 toward 12–16 (the operator's “6–12” upper bound plus headroom) requires a batch-size increase to avoid the “spread same throughput across more queues” failure mode. - Acceptance: a sizing memo with the recommended `--batch-size/--ubatch-size/--parallel` triple for a 12-agent target, to be applied in the same batched coder-pause window as 1.2.

Task 1.4 — Broker `ClassAgent` addition (D5 mitigation). - File:line scope: `submodules/helix_llm/internal/vrambroker/broker.go` — add `ClassAgent Class = "agent"` to the `Class` const block (after `ClassCoder`, line ~20); it is a warm-tier class (falls through to the existing warm-tier `Acquire` path

at `broker.go:151-171`, no `IsBurst()/IsResident()` change needed since the zero-value behavior for a new class not matching either predicate already routes it through the warm-tier admission path correctly — confirm this by reading `IsBurst()/IsResident()` again against the new constant before writing the change, since `broker_test.go/mutation_test.go` already exist and must be extended, not replaced). - Acceptance (§11.4.108 runtime signature): new unit test in `broker_test.go` asserting `ClassAgent.IsResident() == false`, `ClassAgent.IsBurst() == false`, and an `Acquire(ctx, ClassAgent, N)` call is admission-gated by measured free VRAM identically to `ClassVLM` (reuse the existing `newWithReader` test seam, §11.4.27(A) mocks-in-unit-tests- only). Paired §1.1 mutation: temporarily make `ClassAgent.IsResident()` return `true` and assert the new test FAILs (proves the test actually distinguishes resident from warm).

Task 1.5 — Pre-boot config validator (D1/D4/D7 mitigation). - Scope: a new small Go tool/script (location: `submodules/helix_llm/cmd/` per the existing `cmd/videogen-boot`, `cmd/imagegen-boot` pattern — reuse that pattern per §11.4.74) that, given a proposed lane's launch flags (`--ctx-size`, `--parallel`, `-ngl`, quant, model size), computes: (i) worst-case static KV ceiling via the §4 formula, (ii) whether `-ngl 99` full-GPU-residency is achievable given the broker's grantable lease size for that class, and (iii) whether the port/container-name is distinct from every other currently-registered lane. Refuses (non-zero exit) if any check fails. - Acceptance: a Challenge test (§11.4.6 project rule) that feeds the validator a deliberately over-budget Lane B config and asserts non-zero exit + a specific error message identifying which of (i)/(ii)/(iii) failed; a second Challenge feeds a valid config and asserts zero exit.

Phase 2 — Lane B build-out (parallel-safe, no operator gate; depends on Task 1.4/1.5)

Task 2.1 — Lane B model benchmarking spike. - **Candidate list + priority corrected 2026-07-08** (§1(c), independent-review remediation): candidates are now, in priority order, (1) Mistral-Nemo-12B, (2) GLM-4.7-Flash, (3) DeepSeek-Coder-V2-Lite, (4) a second Qwen3-Coder-30B-A3B replica (demoted — does not fit the corrected budget below without an aggressive quant/ctx cut), (5) Hermes-2-Pro's structured-output role remains unfilled/UNCONFIRMED. Renumbers the earlier draft's list; see §1(c) for full VRAM-footprint citations per candidate. - **Mandatory pre-boot go/no-go check (new, §11.4.108/§11.4.110 — do NOT skip):** before booting ANY candidate, re-run `nvidia-smi --query-gpu=memory.total,memory.used,memory.free --format=csv` (read-only) and recompute `free - 2 GiB headroom` = the live Lane-B budget; compare the candidate's actual weight size at the quant to be tested against that budget and require **≥ 2 GiB** remaining for KV cache + activation buffers before proceeding — if the remainder is below that floor (as §1(c) found for GLM-4.7-Flash's smallest

quant, ~0.61 GiB, and for DeepSeek-Coder-V2-Lite's Q4_K_M, ~0.74 GiB, against the 2026-07-08 reading of ~10.39 GiB), the benchmark MUST run with `--parallel 1` and a small `--ctx-size` (single-slot mode) and the report MUST say so explicitly rather than silently degrading to fewer slots than the “lightly-loaded second lane” goal implies. This check itself is the D1/D7 mitigation applied at benchmark time, not just at Task 1.5's config-validator time. - Action: boot Mistral-Nemo-12B (candidate 1, §1c) at Q4 (~6.52 GiB weights) in its OWN container (D4-compliant: distinct port, e.g. 18445) via the containers submodule, gated through the new ClassAgent broker admission (Task 1.4) + the Task 1.5 validator. Benchmark tok/s and tool-calling correctness against a small fixed prompt set (reuse or extend the D6 regression test's prompt set); its function-calling support must be verified directly against this plan's actual tool-schema, not assumed from third-party listicles per §11.4.6. - Acceptance (§11.4.108): a benchmark report (`docs/qa/helixllm_phase2_laneb_<ts>/`) comparing Mistral-Nemo-12B vs. GLM-4.7-Flash (candidate 2) vs. DeepSeek-Coder-V2-Lite (candidate 3) on tok/s, VRAM resident size, KV-headroom-at-benchmarked-quant (cite the go/no-go arithmetic above per candidate), and tool-call correctness rate, with a GO/NO-GO recommendation for which becomes the production Lane B model. A second Qwen3-Coder-30B-A3B replica (candidate 4) is deprioritized per §1(c)'s corrected budget and need only be benchmarked if all three higher-priority candidates fail on tool-calling correctness. Candidate 5 (Hermes-2-Pro's ≤8B structured-output-specific role) remains explicitly marked UNCONFIRMED/deferred per §1(c) — do not implement without a dedicated follow-up research pass identifying a real, current, ≤8B, verified-tool-calling model for that narrower role (§11.4.6: guessing a replacement for Hermes-2-Pro is forbidden without evidence).

Task 2.2 — Lane B production container + broker wiring. - Action: once Task 2.1's GO model is chosen, wire its container into HelixLLM's existing boot/health pattern (mirroring `cmd/videogen-boot/main.go`'s structure per §11.4.74), calling `vrambroker.Acquire(ctx, ClassAgent, needBytes)` before starting the llama-server process, and `Release()` on shutdown. - Acceptance: integration test (real broker + real container boot, no mocks per §11.4.50(A)) asserting: Lane B fails to boot when Lane A + a synthetic over-budget request would exceed free VRAM (`ErrBudgetExceeded` surfaces cleanly, no silent degrade); Lane B boots successfully and serves a real completion when budget is available; `Release()` frees the lease and a subsequent `Budget()` call reflects it.

Phase 3 — Runtime observability + regression coverage (parallel-safe)

Task 3.1 — /metrics-based dashboard/alerting for D1/D2/D3. - Action: a lightweight sidecar (or extension of an existing HelixLLM monitoring component — check submodules/helix_llm for an existing metrics-scrape pattern before writing new, §11.4.74) polling both lanes' `/metrics` + `nvidia-smi` on an interval,

logging KV-utilization %, tok/s, and combined VRAM to a rolling log; alert threshold configurable. - Acceptance: a chaos test (§11.4.85) that saturates Lane A to 8/8 slots and asserts the dashboard's KV-utilization metric crosses the alert threshold within N seconds, captured as evidence.

Task 3.2 — D6 tool-calling-under-load regression guard (permanent, §11.4.135). - Action: implement the reproduce-first test described in D6's mitigation as a permanent registered regression guard, not a one-off. - Acceptance: RED-on-(hypothetically-reverted-GGUF) / GREEN-on-current polarity test per §11.4.115, registered into HelixLLM's standing regression suite.

Task 3.3 — D4/D7 config-drift Challenge (permanent). - Action: a Challenge script that periodically (or on-demand) re-runs Task 1.5's validator against the CURRENTLY running lanes' actual launch configs (via Task 0.1's inspect mechanism), catching drift if a future change bypasses the validator. - Acceptance: Challenge asserts zero drift on a known-good state; paired mutation deliberately misconfigures a lane and asserts the Challenge catches it.

Phase 4 — Operator-gated Lane A changes (SINGLE batched coder-pause window, D8)

Task 4.1 — Apply --kv-unified (Task 1.2) + tuned --batch-size/--parallel (Task 1.3) + GGUF re-pull if needed (Task 1.1) to the live helixllm-coder container, in one window. - Precondition: operator-authorized coder-pause (§11.4.122) — this is an OPERATOR-BLOCKED item per RESUME.md's existing convention for coder-container changes, NOT autonomously takeable; enumerate the unblock choice per §11.4.148(D3): “[A] operator authorizes a single coder-pause window now to apply all three batched changes · [B] defer until the next scheduled maintenance window · [C] apply only the GGUF re-pull (if Task 1.1 found it necessary) and defer the flag changes.” - Acceptance (§11.4.108/§11.4.130): re-run Task 0.2's throughput baseline harness against the restarted container and assert (i) no regression vs. the Phase-0 baseline at 1-agent and 8-agent load, (ii) a measured improvement at 12-agent load specifically attributable to --kv-unified + batch-size tuning (the whole point of this phase), (iii) the D6 regression guard (Task 3.2) still passes post-restart on the new artifact (§11.4.139 clean-artifact runtime signature).

Sources verified 2026-07-08

- llama.cpp Architecture / production inference writeup (continuous batching, slot pool) — <https://markaicode.com/architecture/llamacpp-architecture/>
- llama.cpp System Design (1,000 concurrent users architecture) — <https://markaicode.com/architecture/llamacpp-system-design-architecture-1158/>

- Tuning llama-server on Apple Silicon (parallel/batch-size guidance, cross-referenced for the general parallel-vs-batch-size tradeoff) — <https://medium.com/@michael.hannecke/tuning-llama-server-on-apple-silicon-9b3e778ab100>
- llama.cpp Parallelization/Batching Explanation, GitHub Discussion #4130 — <https://github.com/ggml-org/llama.cpp/discussions/4130>
- Optimal parameters for parallel inference using llama-server, GitHub Discussion #18308 — <https://github.com/ggml-org/llama.cpp/discussions/18308>
- llama.cpp tools/server README (canonical flag reference) — <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>
- llama-server(1) Debian manpage — <https://manpages.debian.org/unstable/llama.cpp-tools/llama-server.1.en.html>
- Host-Memory Prompt Caching in llama-server, GitHub Discussion #20574 — <https://github.com/ggml-org/llama.cpp/discussions/20574>
- Why Your Local LLM Is Slow — llama.cpp Config Guide — <https://omniforge.online/blog/your-local-llm-is-slow-because-of-five-config-flags>
- Set max context length per request independently of unified kv cache total size, GitHub Discussion #22658 — <https://github.com/ggml-org/llama.cpp/discussions/22658>
- Llama Server Context Length Behavior Explained — <https://ventusserver.com/context-length-behavior-explained/>
- KV cache reuse with llama-server, GitHub Discussion #13606 — <https://github.com/ggml-org/llama.cpp/discussions/13606>
- Llama server Context Size, memory and parallelism — https://alejandro.criadoperez.com/blog/Llama_server_memory
- unsloth/GLM-4.7-Flash-GGUF HF discussion — new kv-unified flag + RTX 4090D 48GB perf report — <https://huggingface.co/unsloth/GLM-4.7-Flash-GGUF/discussions/18>
- Qwen3-Coder run-locally tutorial (tool-calling fix confirmation) — <https://unsloth.ai/docs/models/tutorials/qwen3-coder-how-to-run-locally>
- Qwen/Qwen3-Coder-30B-A3B-Instruct model card — <https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct>
- unsloth/Qwen3-Coder-30B-A3B-Instruct-GGUF — <https://huggingface.co/unsloth/Qwen3-Coder-30B-A3B-Instruct-GGUF>
- QwenLM/Qwen3-Coder GitHub repo — <https://github.com/QwenLM/Qwen3-Coder>
- qwen3-coder-30B-A3B tool-calling failure report, anomalyco/opcode issue #1809 — <https://github.com/anomalyco/opcode/issues/1809>
- GLM-4.7-Flash: How To Run Locally, Unsloth docs — <https://unsloth.ai/docs/models/glm-4.7-flash>
- unsloth/GLM-4.7-Flash-GGUF HF repo — <https://huggingface.co/unsloth/GLM-4.7-Flash-GGUF>

- unsloth/GLM-4.7-GGUF (non-Flash, for size/scale contrast) — <https://huggingface.co/unsloth/GLM-4.7-GGUF>
- GLM-4.7-Flash Complete Guide 2026 — <https://aicybr.com/blog/glm-4-7-flash-complete-guide>
- Best DeepSeek Models for Hermes Agent (DeepSeek-V4 2026 context) — <https://www.remoteopenclaw.com/blog/best-deepseek-models-for-hermes>
- Best Open-Source Models for Hermes Agent (Hermes-2-Pro superseded-status context) — <https://www.remoteopenclaw.com/blog/best-opensource-models-for-hermes>
- Hermes 2 Pro Mistral 7B model card — <https://telnyx.com/llm-library/hermes-2-pro-mistral-7b>
- Running Multiple Local Models: Memory Management Strategies — <https://www.sitepoint.com/multiple-local-models-memory-management/>
- How to Run Multiple Models on One GPU — <https://www.aimadetools.com/blog/run-multiple-models-one-gpu/>
- llama-server Shared Alias for multiple instances of the same model in Router Mode, GitHub Discussion #22823 — <https://github.com/ggml-org/llama.cpp/discussions/22823>
- Prior session's own research (cited, not re-verified this session — see that doc's own §8 for its sources): docs/research/07.2026/01_local_models_serving/01_local_models_serving.md
- Live production state (direct source, not web): docs/research/07.2026/00_master/RESUME.md
- Direct source reads (not web): submodules/helix_llm/internal/vrambroker/broker.go, budget.go, submodules/helix_llm/internal/control/scheduler.go

Sources added 2026-07-08 (this fix session — independent-review remediation)

- 15 Best Local LLM Models to Run in 2026, PocketLLM (Mistral-Nemo-12B #4/15, DeepSeek-Coder-V2-Lite #7/15, ranked scores + VRAM figures) — <https://pocketllm.app/blog/best-local-llm-models-2026/>
- 8 Best Local LLMs for Coding in 2026, PocketLLM (DeepSeek-Coder-V2-Lite #2/8, HumanEval + VRAM detail) — <https://pocketllm.app/blog/best-local-llm-for-coding-2026/>
- bartowski/DeepSeek-Coder-V2-Lite-Instruct-GGUF quant-size table (Q4_K_M = 10.36 GB) — cited via search result, HF repo bartowski/DeepSeek-Coder-V2-Lite-Instruct-GGUF
- Mistral VRAM Requirements (2026) — Will Your GPU Run 7B, Nemo 12B, Codestral 22B or Small 24B? — <https://willitrunai.com/blog/mistral-models-gpu-requirements>
- Mistral Nemo 12B: What It Is and When to Use It — <https://mljourney.com/mistral-nemo-12b-what-it-is-and-when-to-use-it/>

- Mistral Nemo 12B: 8GB VRAM, 128K Context, Ollama Setup 2026 — <https://localaimaster.com/models/mistral-nemo-12b>
- `nvidia-smi --query-gpu=memory.total,memory.used,memory.free --format=csv` — direct read-only host command, not web, run this session (2026-07-08) to obtain the current authoritative free-VRAM figure (32607/19436/12685 MiB total/used/free) used throughout §1(c)/§1(f)/Task 2.1’s corrected VRAM math.
- Independent review (this fix session’s input, not web): `scratchpad/review_serving_plan_v2.md` — source of the four findings corrected in this revision.

UNCONFIRMED (excluded from firm recommendations): exact GGUF commit/revision date for the Qwen3-Coder-30B-A3B tool-calling fix vs. the currently-running container’s revision (Task 1.1 needs a direct HF-API check to resolve); GLM-4.7-Flash’s exact context-window ceiling (128K vs. 200K claims conflict across sources); any concrete ≤8B model to replace Hermes-2-Pro’s **pure structured-output-specific** role (narrower framing per this revision’s §1(c) item (5) — explicitly deferred, not guessed, per §11.4.6; Mistral-Nemo-12B and DeepSeek-Coder-V2-Lite are NOT proposed for this narrower role, only for the general Lane-B coding/agent-overflow role); DeepSeek-Coder-V2-Lite’s Q3/Q2 quant sizes (not located this session — needed to determine whether a smaller quant relieves the ~0.74 GiB KV-headroom squeeze found at Q4_K_M); Qwen3.5/Qwen3.6/GLM-4.7(non-Flash)/GLM-5/DeepSeek-V4-fits-32GB claims (all unconfirmed, none actionable here). **Retracted this session (no longer unconfirmed/asserted-false):** the earlier draft’s claim that DeepSeek-Coder-V2-Lite “does not appear in current 2026 discourse” (directly contradicted by evidence, see §0/§1(c)) and that Mistral-Nemo-12B is blanket “2024-superseded” (partially contradicted — it retains a real, currently-recommended VRAM-constrained niche, see §0/§1(c)).